

Introduction

Every developer eventually meets a scheduling problem in disguise. Sometimes it's literally about calendars — meetings, job offers, flight bookings. Sometimes it's a resource allocation problem wearing a different costume. LeetCode 3414, Maximum Score of Non-overlapping Intervals, is one of the clearer examples of why interviewers love this family of problems: it looks like it wants a greedy answer, and greedy answers are exactly wrong here.

This problem matters for interview prep because it tests three separate skills at once. First, whether you recognize that "pick the best-looking option first" is a trap. Second, whether you can turn a scheduling constraint into a sorted, binary-searchable structure. Third, whether you can extend a classic DP (weighted interval scheduling) into a bounded-count version, where you're not just optimizing a total, but optimizing a total under a hard cap on how many items you can choose. That bounded-count twist is what separates this from the textbook version of the problem, and it's the part most people get wrong on a first attempt.

If you've done weighted job scheduling before, this will feel familiar with one extra dimension bolted on. If you haven't, this is a great problem to learn it from, because the constraint (at most 4 picks) forces you to think about *why* the DP needs an extra axis instead of just accepting it as boilerplate.

Problem Overview

You're given a list of intervals. Each interval has three numbers: a start point, an end point, and a weight. You need to choose a subset of these intervals — at most four of them — such that no two chosen intervals overlap. Overlap here is strict: if one interval ends exactly where another begins, they still count as touching, and touching is not allowed.

Your goal is to maximize the sum of the weights of the intervals you pick. If multiple different subsets achieve the same maximum score, you must return the one whose original indices form the lexicographically smallest list when sorted — in practice, this usually just means: among tied options, don't take intervals you don't need.

The output is the list of original indices (not the intervals themselves) of the chosen subset, sorted in ascending order.

Example

Consider six intervals, indexed 0 through 5, with these `[start, end, weight]` values:

Index Start End Weight

0	1	3	2
1	4	5	2
2	1	5	5
3	6	9	3
4	6	7	1
5	8	9	1

Interval 2 runs from 1 to 5 and has the highest single weight on the list, 5. Interval 3 starts at 6, which is strictly after interval 2 ends at 5, so they don't overlap. Together, they sum to $5 + 3 = 8$. Checking every other combination — including ones that squeeze in more than two intervals — nothing beats 8. So the answer is `[2, 3]`.

Notice something important here: the two intervals with the *individually* highest weights aren't necessarily the answer. What matters is which sets of non-overlapping intervals are even reachable, and among those, which sums the highest.

Intuition

Here's the trap version of this problem, and it's worth sitting with because it's exactly how you'd get burned in an interview.

Imagine five job offers on a calendar, each with a start day, end day, and payout. One offer pays `\$500`, dramatically more than the others. Your first instinct is to lock that one in immediately. But the `\$500` job overlaps with two other jobs — say, one paying `\$300` and one paying `\$300` — which together total `\$600`. Skip the flashy `\$500` job, take those two instead, and you've made `\$100` more. The "obviously correct" first move was actually a `\$100` mistake.

This is the core lesson: **the biggest single weight doesn't matter — the biggest chain of compatible weights does**. Greedily grabbing the highest-value interval ignores what it blocks you from taking later. You need a way to reason about *sequences* of compatible intervals, not individual intervals in isolation.

So how does an experienced engineer approach this? Start by sorting all intervals by their end time. Why end time and not start time? Because once you fix an interval, the question you always need to answer is "what's the best score I could have already banked using intervals that finished before this one starts?" Sorting by end time makes that question well-defined and lets you build it up incrementally, similar to how you'd build up a running maximum in classic weighted interval scheduling.

Once you have things sorted, for each interval you need to quickly find the last (or, working forward, the *next*) interval that doesn't conflict with it. Checking every other interval for compatibility is an $O(n)$ operation per interval, which becomes $O(n^2)$ overall — far too slow when n can reach 50,000. Since everything is sorted, this becomes a binary search: find the first interval whose start time is strictly after the current interval's end time. That's one $O(\log n)$ lookup instead of a linear scan.

The final wrinkle is the "at most 4" constraint. In plain weighted interval scheduling, you track one running best score per position. Here, because you're capped at four picks, one number isn't enough — you need to know the best score achievable with exactly 1, 2, 3, or 4 remaining picks from any given position. That turns your DP array into a DP table with an extra dimension: rows for positions, and five columns for "0 through 4 picks."

Brute Force Solution

Idea: Try every possible subset of intervals of size 0 through 4, check whether each subset is pairwise non-overlapping, and keep the maximum-weight valid subset.

Algorithm: 1. Generate all combinations of intervals of size 1, 2, 3, and 4. 2. For each combination, check every pair for overlap. 3. Track the maximum weight, and among ties, the lexicographically smallest index list.

Advantages: Trivial to reason about correctness. No sorting, no DP table, no binary search — just enumeration.

Disadvantages: Choosing up to 4 intervals out of n gives roughly $O(n^4)$ combinations, and checking each combination for pairwise overlap adds further cost. With n up to 50,000, this is astronomically slow — completely infeasible.

Time complexity: $O(n^4)$ or worse. **Space complexity:** $O(1)$ beyond bookkeeping for the current best subset.

This approach only exists as a mental baseline — it will not pass on the actual constraints, but it's useful for confirming correctness on tiny hand-built examples before trusting the optimized version.

Optimal Solution

The optimal approach is a dynamic programming solution built on top of sorting and binary search. Here's the full sequence of steps.

Step 1 — Sort by start time, but remember original positions. Since the final answer needs original indices, sort the interval *indices* by start time rather than sorting the intervals themselves. Then pull out three parallel arrays — `start` , `end` , `weight` — in that sorted order.

Step 2 — Precompute the "next compatible interval" for each position. For every interval `i` in sorted order, binary search the `start` array for the first index whose start time is strictly greater than `end[i]` . Call this `nxt[i]` . This is the earliest interval you could take right after taking interval `i` .

Step 3 — Build the DP table. Create a table `dp` with `n + 1` rows and 5 columns (for 0 through 4 picks remaining). The extra row, `dp[n]` , represents "no more intervals left," and is all zeros. Fill the table from the bottom (last interval) up to the first:

- `dp[i][0] = 0` for every `i` (zero picks left means zero score, no matter what).
- For `k` from 1 to 4: `dp[i][k] = max(dp[i+1][k], weight[i] + dp[nxt[i]][k-1])` .

The first term is "skip interval `i` , keep whatever the best score was without it." The second term is "take interval `i` , add its weight, and jump ahead to the best score reachable from its next compatible interval, using one fewer pick." Take the larger of the two.

Step 4 — Read off the answer. The maximum possible score is `dp[0][4]` — the best score achievable starting from the very first interval with all four picks still available.

Step 5 — Reconstruct the actual chosen intervals. Walk forward from `i = 0` , `k = 4` . At each step, compare "take" versus "skip" the way the DP did. If taking *strictly* beats skipping, record this interval's original index, jump to `nxt[i]` , and decrement `k` . Otherwise, move to the next interval in sorted order without spending a pick. The word "strictly" matters: on an exact tie, skipping keeps the result using fewer indices, which is what guarantees the lexicographically smallest answer when multiple subsets tie for the top score.

Since indices get collected in whatever order the chain happens to walk them (not necessarily increasing), sort the collected list before returning it.

Here's a small trace of the table shape for the earlier six-interval example (values simplified for illustration):

i (sorted) k=0 k=1 k=2 k=3 k=4

5	0	1	1	1	1
4	0	1	1	1	1
3	0	3	4	4	4
2	0	5	8	8	8
1	0	5	8	8	8

i (sorted) k=0 k=1 k=2 k=3 k=4

0 0 5 8 8 8

`dp[0][4] = 8` , matching the expected answer.

Python Code

```
from bisect import bisect_right
from typing import List

def maximum_weight(intervals: List[List[int]]) -> List[int]:
    n = len(intervals)

    # Sort original indices by start time.
    order = sorted(range(n), key=lambda i: intervals[i][0])
    start = [intervals[i][0] for i in order]
    end = [intervals[i][1] for i in order]
    weight = [intervals[i][2] for i in order]

    # nxt[i]: first sorted index whose start is strictly after end[i].
    nxt = [bisect_right(start, end[i]) for i in range(n)]

    # dp[i][k]: best score using intervals[i:], picking at most k of them.
    dp = [[0] * 5 for _ in range(n + 1)]
    for i in range(n - 1, -1, -1):
        for k in range(1, 5):
            skip = dp[i + 1][k]
            take = weight[i] + dp[nxt[i]][k - 1]
            dp[i][k] = max(skip, take)

    # Reconstruct the lexicographically smallest set of original indices.
    chosen = []
    i, k = 0, 4
    while i < n and k > 0:
        skip = dp[i + 1][k]
        take = weight[i] + dp[nxt[i]][k - 1]
        if take > skip:
            chosen.append(order[i])
            i = nxt[i]
            k -= 1
        else:
            i += 1

    return sorted(chosen)

def demo() -> None:
    intervals = [[1, 3, 2], [4, 5, 2], [1, 5, 5], [6, 9, 3], [6, 7, 1], [8, 9, 1]]
    result = maximum_weight(intervals)
```

```

assert result == [2, 3], result
print("ok:", result)

if __name__ == "__main__":
    demo()

```

Code Walkthrough

- `order = sorted(range(n), key=lambda i: intervals[i][0])` sorts indices, not the intervals themselves, so original positions are never lost.
- `start`, `end`, `weight` are parallel arrays reindexed into sorted order — this keeps every later lookup a plain array access instead of repeated dictionary or tuple unpacking.
- `nxt[i] = bisect_right(start, end[i])` finds the first sorted position whose start time is strictly greater than `end[i]`. Because "touching" counts as overlapping, this needs to be strictly greater, which is exactly what `bisect_right` gives against a value equal to `end[i]`.
- The DP table is built backwards (`range(n - 1, -1, -1)`) because `dp[i][k]` depends on `dp[i+1][k]` and `dp[nxt[i]][k-1]`, both of which refer to positions *after* `i`.
- `dp[i][0]` is implicitly always 0 since the table is initialized with zeros and the loop only touches columns 1 through 4.
- The reconstruction loop mirrors the DP recurrence exactly, so whatever choice built the table is retraced faithfully — this symmetry is what keeps the reconstruction correct.
- `chosen.append(order[i])` translates a sorted-order position back into its original index before storing it.
- The final `sorted(chosen)` step exists purely because indices get collected in traversal order, which is not the same as ascending numeric order.

Dry Run

Using the six-interval example: `order` sorts by start time, which for this example keeps positions 0–5 in the same order already (starts are 1, 4, 1, 6, 6, 8 → after a stable sort by start, index 2 comes right after index 0 since both start at 1, but the exact tie-break doesn't change the outcome here).

Working backwards, `dp[5][k] = 1` for `k ≥ 1` (only interval 5 available, weight 1). `dp[4][k]` compares skipping (inherits from `dp[5]`) versus taking interval 4 (weight 1, jumping past it) — they tie, so `dp[4][k] = 1`. `dp[3][k]` for `k ≥ 1` considers taking interval 3 (weight 3) jumping to `nxt[3]`, versus skipping — taking wins, giving `dp[3][k] = 3` for `k=1` and higher once compounded with later picks (works out to 4 once interval 5 is added at `k=2`). `dp[2][k]` considers

taking interval 2 (weight 5) and jumping to `nxt[2] = 3` (interval 3 starts at 6, which is after interval 2 ends at 5): `5 + dp[3][k-1]`. At `k=4`, that's `5 + dp[3][3] = 5 + 3 = 8`, beating skipping.

Reconstruction starts at `i=0, k=4`. Interval 0 (weight 2) doesn't beat skipping, so move to `i=1`. Interval 1 also doesn't beat skipping, move to `i=2`. Interval 2's "take" value (`5 + dp[3][3] = 8`) beats "skip" (`dp[3][4]`, which is also 8 in this table but not strictly greater — actually since take equals or exceeds skip and the recurrence favored take when building `dp[2][4]=8`, the reconstruction must match `dp[2][4]` to decide; taking interval 2 does achieve the recorded optimum), so index 2 is recorded, jump to `nxt[2] = 3, k=3`. At `i=3, k=3`, interval 3 (weight 3) beats skipping, so index 3 is recorded, jump to `nxt[3]` (past interval 3), `k=2`. From there, nothing further improves the score, so the loop ends. Final result: `sorted([2, 3]) = [2, 3]`, matching the expected output.

Complexity Analysis

Time: Sorting indices is $O(n \log n)$. Computing `nxt[i]` for every interval via binary search is $O(n \log n)$ total. Building the DP table is $O(n)$ rows times a constant 5 columns, so $O(n)$. Reconstruction is $O(n)$ in the worst case. Total: **$O(n \log n)$** , dominated by sorting and binary search.

Space: The `start`, `end`, `weight`, and `nxt` arrays are each $O(n)$. The DP table is $O(n)$ rows times a constant 5 columns, so $O(n)$ as well. Total: **$O(n)$** .

These bounds are correct because the constant factor of 5 (from "at most 4 picks") never scales with input size — it's a fixed cap from the problem statement, not a variable dimension.

Alternative Solutions

A segment tree or Fenwick tree keyed on coordinate-compressed end times could replace the binary search step, letting you query "best score ending before position x" more flexibly if the problem allowed more general range queries. For this specific problem, that's unnecessary complexity — binary search on a sorted array does the same job with far less code and the same asymptotic complexity. The DP-with-binary-search approach is the standard, interview-appropriate solution here.

Edge Cases

- **Empty input:** `n = 0` should return an empty list — the DP table degenerates to just the base row of zeros.
- **Fewer than 4 valid non-overlapping intervals exist:** the algorithm naturally picks fewer than 4, since `dp[i][k]` never forces a pick that doesn't improve the score.

- **All intervals overlap each other:** only the single best-weight interval should be selected.
- **Duplicate intervals** (same start, end, and weight): both are valid candidates independently; the tie-breaking rule (prefer skipping on equal score) determines which index is picked.
- **Touching endpoints** (one interval's end equals another's start): must be treated as overlapping, not compatible — this is the single most common source of off-by-one bugs in this problem.
- **Maximum-size input** ($n = 50,000$) with large coordinate values (up to 10^9): confirms the solution doesn't rely on indexing into a day-by-day array, which would be infeasible at this scale.

Common Mistakes

1. **Treating touching intervals as compatible.** If interval A ends at 5 and interval B starts at 5, they still overlap — the binary search must use strictly-greater-than logic (`bisect_right` , not `bisect_left` , against the right value).
2. **Using a single running-best array instead of a 5-column DP table.** This silently ignores the "at most 4 picks" cap and reduces to unbounded weighted interval scheduling.
3. **Greedily sorting by weight instead of end time.** This is the exact trap the problem is designed around — it produces plausible-looking but wrong answers.
4. **Scanning linearly for the next compatible interval** instead of binary searching, which passes small tests but times out at $n = 50,000$.
5. **Taking on a tie during reconstruction** instead of skipping, which produces a valid-scoring but not lexicographically smallest index list.
6. **Forgetting to sort the final index list**, since indices are collected in whatever order the DP chain visits them, not ascending order.
7. **Sorting the intervals themselves instead of their indices**, losing the mapping back to the original positions needed for the answer.

Interview Questions

- Why does sorting by end time (rather than start time) make the DP recurrence well-defined?
- How would the solution change if the cap were a general `k` instead of a fixed 4?
- What's the worst-case time complexity if you replaced binary search with a linear scan, and why does the fixed cap of 4 not save you there?
- How do you guarantee the lexicographically smallest index set among ties, and why does "skip on exact tie" achieve that?
- Could this be solved with a greedy algorithm under any additional constraints? If so, which ones?

Similar Problems

- **LeetCode 435 — Non-overlapping Intervals:** The unweighted, unbounded-count ancestor of this problem; a good warm-up for understanding overlap conditions.
- **LeetCode 1235 — Maximum Profit in Job Scheduling:** The classic unbounded weighted interval scheduling problem this one builds on, minus the "at most k" cap.
- **LeetCode 300 — Longest Increasing Subsequence:** Shares the "binary search plus DP" pattern for reducing an $O(n^2)$ recurrence to $O(n \log n)$.
- **LeetCode 1751 — Maximum Number of Events That Can Be Attended II:** Nearly identical structure — weighted interval scheduling capped at k selections, using the same DP-plus-binary-search template.

Key Takeaways

The biggest single weight is not the same as the biggest reachable chain — greedy choices that ignore downstream consequences will cost you. Sorting by end time and binary searching for the next compatible interval turns an $O(n^2)$ scheduling problem into $O(n \log n)$. When a problem caps *how many* items you can select, that cap becomes a new dimension in your DP table, not an afterthought bolted onto a single-value DP. And when correctness requires a specific tie-breaking rule, encode it directly into the reconstruction logic — don't leave it to chance.

Watch the Video

For a visual walkthrough of the binary search jumps and the DP table filling in step by step, watch the full breakdown here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series dedicated to making coding interview problems and core computer science concepts genuinely intuitive rather than memorized. Every video works through the reasoning an experienced engineer actually goes through — the false starts, the "wait, why doesn't the obvious approach work" moments, and the pattern recognition that gets you to the right data structure. The goal isn't just to pass this one problem; it's to recognize the same shape the next time it shows up in a different costume.

Call To Action

If this explanation helped the problem finally click, give the video a like on YouTube and subscribe for more interview-focused breakdowns like this one. For deeper dives and written companions to every video, subscribe to the Substack newsletter. Drop a comment with the problem you'd like broken down next, and share this with anyone prepping for interviews who keeps getting tripped up by greedy-looking DP problems.

Quick Review

When intervals must be non-overlapping but you pick K of them, what's the tell?

Weighted interval scheduling pattern: sort by end time, DP where $dp[i][k]$ = best score using first i intervals, k picks.

Time complexity for K-limited weighted interval scheduling with binary search?

$O(n \log n)$: sort intervals, then for each use binary search to find last non-overlapping predecessor.

Space complexity of the DP table for this problem?

$O(n*k)$ for the dp array; can sometimes trim to $O(k)$ per row but $n*k$ is standard and safe.

Why does greedily grabbing the biggest-score interval first fail?

It can block two smaller overlapping-free intervals whose combined score beats the one big pick — greedy ignores future exclusions.

Weighted interval scheduling vs plain activity selection (max count)?

Activity selection maximizes count and greedy-by-end-time works; weighted scoring needs DP since values differ, greedy breaks.

Interview follow-up: how do you find the last compatible interval fast?

Binary search on sorted end times for the rightmost interval whose end $\leq$ current start.